
Spring Boot

What is Spring Boot ?

Spring Boot is a framework used to build Spring-based applications quickly with minimum configuration.

>> Key Features of Spring Boot

- Auto Configuration
 - Minimal Setup
 - Embedded Server (Tomcat by default)
 - Fast Project Setup
 - Production-ready features (Actuator, Logging, etc.)
-

>> Spring Boot Setup

We can create a Spring Boot project using this type :

>> Spring Initialize

- Select : Maven
- Choose : Java version
- Select dependencies
- Download and Open in IntelliJ

OR

>> IntelliJ (New Project → Spring Initialize)

Same process, directly inside IDE.

>> Common Dependencies

- Spring Web
 - Spring Data JPA
 - Lombok
 - MySQL Driver
 - Spring Boot Dev Tools
 - Spring Boot Starter Validation
 - Spring Security
-

>> Standard Project Structure

src/main/java
com.app

Packages Name : controller | service | repository | entity | dto | exception |
config

>> Spring Boot Concepts : (@SpringBootApplication)

@SpringBootApplication is a single annotation used to start a Spring Boot application.

=====

It is a combination of : @Configuration

-- Marks the class as a Spring configuration class
-- We can define beans inside this class (e.g. using @Bean)

Example

-- @Configuration class = Bean factory
-- @Bean method = Create a Bean Method

=====

>> @EnableAutoConfiguration

-- Spring Boot automatic configures components based on dependencies

Example :

-- Tomcat
-- Data Source
-- JPA / Hibernate
-- Spring Security
-- Jackson JSON

=====

>> @ComponentScan

It is scan all classes in this package that have Spring annotations and register them as beans in the Spring container.

Which annotations does it scan?

-- @Component
-- @Service
-- @Repository
-- @Controller
-- @RestController

=====

>> IOC Container + Dependency Injection

IOC (Inversion of Control Container)

Spring IOC container creates and manages objects (beans).

>> Dependency Injection (DI)

Spring automatic injects required dependencies.

=====

>> What is a Bean?

A Bean is an object that is created and managed by the Spring container and used for dependency injection.

```

-----
--

MUST-KNOW ANNOTATIONS

=====

>> Core Stereotype Annotations

-- This is create a object (bean) of this class and manage it inside the Spring
container.

@Component

-- It makes a normal class a Spring Bean.
-- It is a generic annotation and can be used in any layer.

>> @Service

-- Used for business logic layer
-- Internally same as @Component
-- Improves readability and layer clarity

>> @Repository

-- Used for database layer
-- Handles DAO / Repository classes
-- Converts DB exceptions into Spring readable exceptions

>> @Controller

-- Used in Spring MVC
-- Returns HTML / JSP pages

>> @RestController

-- Used for REST APIs
-- Returns JSON response
-- Combination is this : @Controller | @ResponseBody

=====

>> Dependency Injection Annotations

>> @Autowired : Automatic injects dependencies

Used in :

-- Field Injection ( Directly Variable )
-- Constructor Injection ( Through Constructor )
-- Setter Injection ( Through Setter Method )

>> @Qualifier

-- Used when multiple beans of same type exist.
-- It specifies which implementation should be injected.

>> Constructor Injection

-- Best for testing ( Easy for testing )
-- Makes class immutable because dependency fixed
-- @Autowired is optional in modern Spring Boot

```

=====

>> Configuration Annotations

>> @Configuration

-- Marks class as configuration class (created a beans here)

>> @Bean

-- Converts the return object of a method into a Spring Bean

-- Mostly used for third-party objects (where we don't use annotation)

Example:

-- ModelMapper

-- RestTemplate

-- PasswordEncoder

=====

>> REST API in Spring Boot

Flow Architecture : Client → Controller → Service → Repository → Database

=====

>> HTTP Methods

Method	Use
GET	Read
POST	Create
PUT	Full Update
PATCH	Partial Update
DELETE	Remove

--

>> CORS (Cross-Origin Resource Sharing)

-- Allowing the browser to send requests from a different origin (domain/port).

>> What is an Origin : Origin = protocol + domain + port

-- Example : These are different origins (because the port is different).

Frontend : http://localhost:3000

Backend : http://localhost:8080

>> When does the CORS problem happen ?

-- When : Angular frontend (port 3000) sends a request

-- to Spring Boot backend (port 8080)

-- Then the browser shows : Blocked by CORS policy

>> @CrossOrigin : Allow this frontend to access the backend

-- This annotation is used to enable CORS.

-- @CrossOrigin(origins = "http://localhost:3000")

>> Where can we use @CrossOrigin ?

```

-- At controller class level
-- At method level

-----
--

>> Spring MVC Request Handling

=====

>> @RequestMapping

-- Used for base mapping at controller level or method level.

>> @GetMapping / @PostMapping / @PutMapping / @DeleteMapping

-- Shortcut annotations for HTTP methods

>> @PathVariable

-- Used to get values from the URL path.
-- @GetMapping("/users/{id}") → (id - 111 )

>> @RequestParam

-- Used to get values from query parameters.
-- @GetMapping("/users")

>> @RequestBody

-- Used in POST / PUT to accept JSON input from request body
-- Converts JSON into a Java object automatic.

>> ResponseEntity

-- ResponseEntity is used to return a complete HTTP response from a controller.
-- It allows you to control : Response body | HTTP status code | Headers

Example

-- return ResponseEntity.ok(user);
-- return new ResponseEntity<>("User Created", HttpStatus.CREATED);

Why use it ?

-- Used to return proper HTTP status codes (200, 201, 400, 404, etc.)
-- Used to send professional REST API responses.

>> @RequestHeader

-- Used to get values from the HTTP request headers.

>> Most common use :

-- Getting JWT token from : Authorization header
-- Example : Authorization : Bearer <token>

-----
--

>> DTO ( Data Transfer Object )

-- Returning Entity directly is a bad practice.

```

>> Why DTO ?

- Better Security
- Clean response structure
- Can hide sensitive fields
- Can add extra response fields

>> DTO → Entity Mapping

- Converting DTO into Entity (during save/update)
- Converting Entity into DTO (while returning response)

>> Why Mapping is needed ?

- Controller should not use Entity directly
- To get a clean and secure API response

1) Manual Mapping

- We set fields manually
- Example : `user.setName(dto.getName());`

2) ModelMapper (Automatic Mapping)

- Automatically maps DTO → Entity
- Example : `modelMapper.map(dto, User.class);`

--

>> Validation (Very Important)

=====

- Dependency : `spring-boot-starter-validation`

>> Common Validation Annotations

- `@NotNull` (Value can't be null)
- `@NotBlank` (String is not empty or blank)
- `@Size` (Length range can be define [min , max])
- `@Email` (Checked valid email format)
- `@Min , @Max` (Set number range)

Example :

```
public class UserDTO {  
  
    @NotBlank  
    private String name;  
}
```

- `@NotBlank` means the value cannot be null
- Cannot be empty (`""`), and cannot contain only spaces (`" "`).

Controller :

```
public ResponseEntity<?> addUser(@Valid @RequestBody UserDTO dto)
```

- `@RequestBody` → Converts the JSON request body into a UserDTO object.
- `@Valid` → Checks the validation rules defined in the DTO.
- If validation fails → Spring automatically returns 400 Bad Request.

```
-----
--

Exception Handling

=====

>> Global Exception Handler

-- It is a central class that handles exceptions for the whole application.
-- We don't need to write try-catch in every controller.
```

Use :

@RestControllerAdvice

```
-- Makes the class a global exception handler.
-- It handles exceptions from all controllers.
```

@ExceptionHandler

```
-- Used to handle a specific exception type.
-- When that exception occurs the mapped method runs.
```

Handles :

```
-- UserNotFoundException → when a user is not found
-- Validation exceptions → when @Valid fails
-- Generic exceptions → any unexpected error
```

```
-----
--
```

>> Spring Data JPA (Database)

=====

>> Entity

```
@Entity
@Table(name="users")
public class User { }
```

```
-- @Entity → This class represents a database table.
-- @Table(name="users") → Maps this entity to the users table.
```

>> Repository

```
public interface UserRepo extends JpaRepository<User, Long> { }
```

```
-- JpaRepository provides all CRUD operations automatic.
-- User → Entity class
-- Long → Primary key type
```

>> Important JPA Methods

```
-- save() → Insert / Update
-- findById() → Get record by ID
-- findAll() → Get all records
-- deleteById() → Delete by ID
-- existsById() → Check if ID exists
```

=====

Custom Queries

>> Derived Queries

-- Spring automatically creates queries based on method name.

-- findByName()
-- findByEmail()

>> JPQL

-- Works on Entity and fields not table and columns.
-- @Query("select u from User u where u.email=?1")

>> Native Query

-- Direct SQL query (table/column based).
-- @Query(value="select * from users", nativeQuery=true)

--

>> Relationships in JPA (Advanced)

>> OneToOne : (One record is linked to exactly one record)

Example : User ↔ Profile : (1 user has 1 profile, and 1 profile belongs to 1 user)

>> OneToMany / ManyToOne : (One record can have multiple related records)

Example : Department → Employees

-- 1 department has many employees, but each employee belongs to 1 department.

>> ManyToMany : (Many records can be linked to many records)

Example : Student ↔ Courses

-- 1 student can take many courses, and 1 course can have many students.

>> Fetch Types

-- LAZY (Recommended)

Data loads only when needed or Better performance.

-- EAGER (Risky)

Data loads immediately with the main entity or Can cause performance issues.

>> Cascade Types

-- PERSIST : If parent is saved, child also gets saved.
-- MERGE : If parent is updated, child also gets updated.
-- REMOVE : If parent is deleted, child also gets deleted.
-- ALL : If you do anything on parent, it will also happen on child.

--

>> Pagination + Sorting

-- Pagination means getting data in pages, not all at once.
-- Sorting means arranging data in an order.

Pageable pageable = PageRequest.of(page, size, Sort.by("name"));

```
-- page → which page you want (starts from 0)
-- size → how many records per page
-- Sort.by("name") → sort the results by the name field (ascending by default)
```

👉 Get a specific page of data, with fixed size, sorted by name.

```
Pageable pageable = PageRequest.of(pageNo, pageSize,
Sort.by("name").ascending());
Page<User> users = userRepo.findAll(pageable);
```

```
-----
--
```

>> Logging in Spring Boot

```
-- Logging means printing useful messages about our application.
-- Example.. what is happening | where the issue is | debugging info | errors
```

Use :

```
-- SLF4J : Logging interface (standard)
-- Logback : Default logging implementation in Spring Boot
```

Log levels :

```
-- INFO : Normal application flow messages
-- Example : User created successfully

-- DEBUG : Detailed messages for debugging
-- Example : Request data : ....
```

```
-- ERROR : Serious problems / exceptions
-- Example : Database connection failed
```

```
-----
--
```

>> Configuration (application.properties / yml)

```
-- This file is used to configure your Spring Boot application, especially
database + JPA settings.
```

Example : application.properties / yml

```
spring.datasource.url=jdbc:mysql://localhost:3306/test
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

=====

```
-- spring.datasource.url           → Database connection URL
-- spring.datasource.username       → DB username
-- spring.datasource.password       → DB password
-- spring.jpa.hibernate.ddl-auto=update → Automatically creates/updates
tables
-- spring.jpa.show-sql=true         → Prints SQL queries in console
```

```
-----
--
```

>> Profiles (Dev / Prod)

-- Profiles are used to keep different configurations for different environments.

-- Dev profile → local database, debugging ON
-- Prod profile → live database, debugging OFF

>> Example

-- application-dev.properties → settings for development
-- application-prod.properties → settings for production

>> Run : properties

-- spring.profiles.active=dev : Then it will load application-dev.properties.
-- spring.profiles.active=prod : Then it will load application-prod.properties.

--

Spring Security (Advanced + Must)

=====

>> Authentication vs Authorization

-- Authentication = Who are you ? (Login check)
-- Authorization = What can you access ? (Role/permission check)

>> Basic Security

-- In Spring Boot, when we add Spring Security dependency, default login is automatic enabled.

>> JWT Authentication (Most Used)

Flow :

-- User logs in → gets JWT token
-- User sends token in Authorization header
-- Security filter validates token
-- If token is valid → user gets access (authorized)

--

Filters + Interceptors (Advanced)

=====

>> Filter

-- A filter runs before Spring processes the request.
-- It works at the Servlet level (before the request enters Spring MVC).

>> Interceptor

-- An interceptor runs inside Spring MVC.
-- It runs before and after the Controller.

--

>> Actuator (Monitoring)

Dependency : spring-boot-starter-actuator

Provides endpoints like:

```
-- /health    → app status (UP / DOWN)
-- /metrics   → performance metrics
-- /beans     → list of Spring beans
-- /env       → environment properties
```

--

>> AOP (Very Advanced)

Use cases :

```
-- Logging
-- Performance monitoring
-- Security
-- Transactions
```

Annotations :

```
-- @Aspect    → marks aspect class
-- @Before     → runs before method
-- @After      → runs after method
-- @Around     → runs before + after (most powerful)
```

--

>> Transactions

```
-- Ensures all DB operations succeed or all rollback (no partial update).
```

Use : Use @Transactional on service methods

Used in :

```
-- Money transfer
-- Multi-step DB operations (save + update + delete together)
```

@Transactional

```
public void transferMoney(Account from, Account to, double amount) {
    from.setBalance(from.getBalance() - amount);
    to.setBalance(to.getBalance() + amount);
}
```

--

Microservices Ready Concepts

=====

>> API Gateway (Client → API Gateway → Microservices)

```
-- Single entry point / main gate for all microservices.
-- The client does not call microservices directly.
```

Handles :

- Routing (sending request to the correct service)
- Security (JWT/Auth checks)
- Rate limiting (limit requests per user)
- Logging and monitoring

=====

>> Eureka Server

- It works like a service registry / directory.

How it works :

- Each microservice registers itself with Eureka.
- Other services or the gateway ask Eureka : Where is user-service running ?

Benefit :

- No hardcoded service URLs
(A hardcoded URL means a URL that is fixed/written directly in the code.
String url = "http://localhost:8082/user-service/users";)

- Supports auto-scaling and dynamic service discovery

=====

>> Config Server

- A central place to store configuration for all microservices.

>> It stores

Database configs | Ports | Keys | Dev/Prod environment properties

Benefit :

- Easy configuration management
- No need to maintain separate config in every service

=====

>> Circuit Breaker (Resilience4j)

- Prevents cascading failures in microservices.

Problem :

- If one service is down (example : Payment service), and other services keep calling it , the whole system can become slow or crash.

>> Circuit Breaker solution

- Stops calling the failing service repeatedly
- Returns a fallback response
- Tries again after some time

Benefit : System stays stable even if one service fails

--

>> Testing

=====

>> Unit Testing

-- Tests individual classes / methods
-- Tools : JUnit | Mockito

>> Integration Testing

-- Tests full Spring Boot flow (Controller → Service → DB)
-- Tools : @SpringBootTest | MockMvc

--

Swagger (API Documentation)

=====

>> Dependency : springdoc-openapi

>> Provides : Swagger UI | API Documentation

--